

System Rozpoznawania Cyfr

1. Opis- wstęp teoretyczny: Projekt wykorzystuje techniki Głębokiego Uczenia a konkretnie Sieci Neuronowe, które są standardem w zadaniach rozpoznawania obrazu.

Kluczowe elementy systemu:

Zbiór danych: System opiera się na zbiorze MNIST, który zawiera 70 000 obrazów cyfr (0-9) w skali szarości o wymiarach 28x28 pikseli.

Architektura CNN: Sieć neuronowa (zdefiniowana jako `MNISTNet`) uczy się hierarchii cech obrazu. W przeciwieństwie do klasycznych sieci MLP (Multi-Layer Perceptron), CNN wykorzystuje operację splotu do wykrywania krawędzi, kształtów i złożonych wzorców, niezależnie od ich położenia na obrazie.

Proces uczenia:

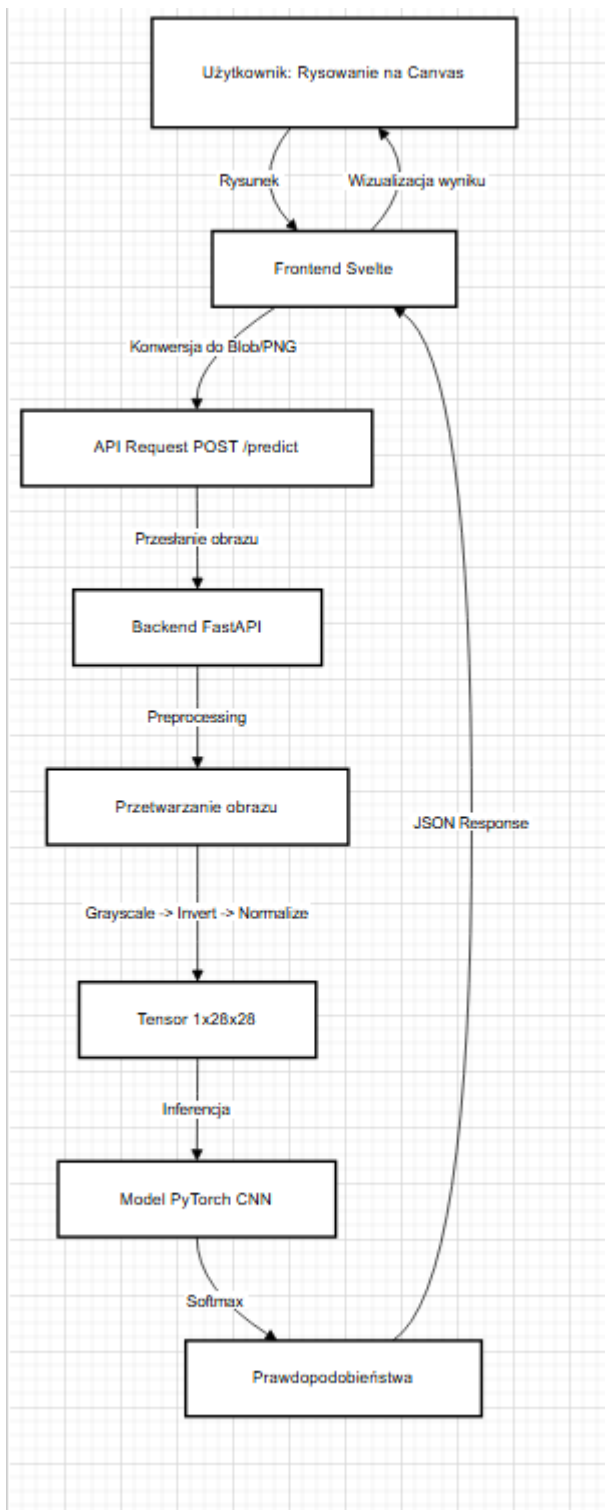
Model jest trenowany przy użyciu algorytmu optymalizacji Adam oraz funkcji straty CrossEntropyLoss.

Zastosowano augmentację danych treningowych (losowe rotacje o 10 stopni, przesunięcia), aby zwiększyć odporność modelu na niedokładności pisma użytkownika i zapobiec przeuczeniu.

Auto-training: System posiada mechanizm wykrywania braku modelu. W takim przypadku serwer automatycznie uruchamia proces treningowy w tle przy pierwszym starcie.

2. Schemat blokowy opisujący projekt

Poniżej przedstawiono przepływ danych w aplikacji, od interakcji użytkownika po predykcję:



3. Fragmenty kodu:

Pętla treningowa (Python - [train_model.py](#)):

```
def train_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for batch_idx, (data, target) in enumerate(loader):
        data, target = data.to(device), target.to(device)

        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = torch.max(output.data, 1)
        total += target.size(0)
        correct += (predicted == target).sum().item()
```

B. Endpoint predykcyjny (Python - [main.py](#))

```
@app.post("/predict")
async def predict(file: UploadFile = File(...)):
    try:
        image = await file.read()
        image_tensor = preprocess_image(image).to(device)

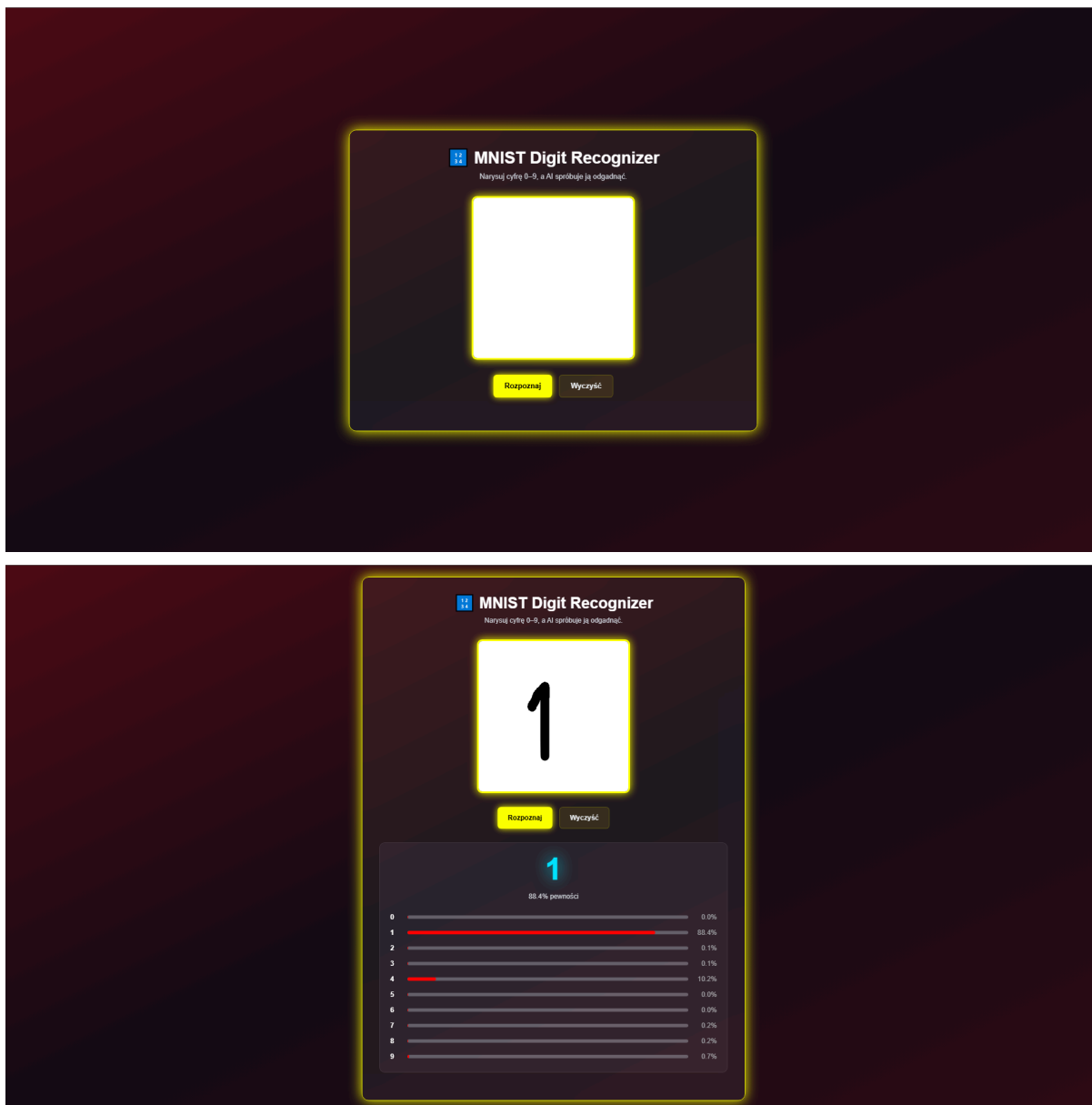
        with torch.no_grad():
            output = model(image_tensor)
            print(f"Raw model output: {output[0].tolist()}")
            probabilities = torch.nn.functional.softmax(output, dim=1)

        probs = probabilities.cpu().numpy()[0]
        predicted_digit = int(np.argmax(probs))
        confidence = float(np.max(probs))

        return {
            "success": True,
            "predicted_digit": predicted_digit,
            "confidence": confidence,
            "probabilities": {
                str(i): float(prob) for i, prob in enumerate(probs)
            }
        }
    except Exception as e:
        return {
            "success": False,
            "error": str(e)
        }
```

4. Interfejs użytkownika:

Interfejs użytkownika został zaimplementowany przy użyciu frameworka **Svelte** oraz biblioteki stylów **Tailwind CSS**.



Kluczowe cechy UI:

- **Design:** Ciemny motyw z neonowymi, żółtymi akcentami oraz tłem z gradientem.
- **Interakcja:**
 - Centralny obszar roboczy reagujący na zdarzenia myszy.
 - Przycisk "Rozpoznaj" wysyłający asynchroniczne żądanie do API.
 - Przycisk "Wyczyść" resetujący płótno i stan aplikacji.
- **Prezentacja wyników:**
 - Duża cyfra reprezentująca główną predykcję.
 - Paski postępu wizualizujące rozkład prawdopodobieństwa dla wszystkich cyfr 0-9.

5. Wnioski i możliwości rozwoju:

Wnioski:

- Integracja FastAPI i Svelte pozwala na stworzenie responsywnej i lekkiej aplikacji AI.
- Automatyzacja procesu treningowego czyni aplikację łatwą w instalacji dla użytkownika końcowego – "plug & play".
- Zastosowanie augmentacji danych znacząco poprawia generalizację modelu na danych rzeczywistych (rysunkach użytkownika).

Możliwości rozwoju:

1. **Wsparcie mobilne:** Dodanie obsługi zdarzeń dotykowych na elemencie Canvas, aby umożliwić rysowanie na smartfonach i tabletach.
2. **Rozszerzenie datasetu:** Zastąpienie MNIST zbiorem **EMNIST**, co umożliwiłoby rozpoznawanie liter.
3. **Optymalizacja modelu:** Eksport modelu do formatu **ONNX** i uruchamianie go bezpośrednio w przeglądarce (onnx.js), co odciążałoby serwer.
4. **Historia operacji:** Dodanie funkcjonalności "Cofnij" (Undo) dla pociągnięć pędzla.